

NAME: HEMANTH REDDY

REG NUMBER: 192472215

COURSE CODE:CSA0910

COURSE NAME:PROGRAMMING IN JAVA

### 1.Single Inheritance – Scenario Scenario:

A company has a Vehicle class with methods startEngine() and stopEngine(). A Car class extends Vehicle and adds a playMusic() method. The company now wants every car to automatically log the engine start time whenever startEngine() is called, while still using the original engine-start functionality

#### Class Diagram

Vehicle

|

|

Car

#### Q1. How would you implement this using single inheritance?

Create a parent class Vehicle containing startEngine() and stopEngine(). Then create a child class Car that extends Vehicle.

```
class Vehicle {  
    void startEngine() {  
        System.out.println("Engine Started");  
    }  
  
    void stopEngine() {  
        System.out.println("Engine Stopped");  
    }  
}  
  
class Car extends Vehicle {  
    void playMusic() {  
        System.out.println("Playing Music");  
    }  
}
```

```
}  
}
```



The screenshot shows an IDE window titled 'Main.java'. The code defines a `Car` class with methods `startEngine()`, `playMusic()`, and `stopEngine()`. A `Main` class contains a `main` method that creates a `Car` object and calls these methods. The output pane on the right shows the execution results: 'Engine Started', 'Playing Music', and 'Engine Stopped', followed by '=== Code Execution Successful ==='.

```
Main.java  
14- void playMusic() {  
15-     System.out.println("Playing Music");  
16- }  
17- }  
18-  
19- public class Main {  
20-  
21-     public static void main(String[] args) {  
22-  
23-         Car car = new Car();  
24-  
25-         car.startEngine(); // Inherited from Vehicle  
26-         car.playMusic();   // Car's own method  
27-         car.stopEngine();  // Inherited from Vehicle  
28-     }  
29- }
```

Output  
^ Engine Started  
Playing Music  
Engine Stopped  
  
=== Code Execution Successful ===

### Explanation:

- Vehicle is the parent class.
- Car is the child class.
- Car automatically inherits `startEngine()` and `stopEngine()`.
- Car also has its own method `playMusic()`.

### Question 2: Should Car override `startEngine()`? Why?

#### Answer

Yes.

- The Car class should override the `startEngine()` method because the company requires every car to log the engine start time whenever the engine starts.
- By overriding the method, the child class can add extra functionality while still performing the original engine-start operation.
- 

### Example

```
class Car extends Vehicle {  
    @Override  
    void startEngine() {  
        super.startEngine();  
        System.out.println("Engine Started at : "  
            + java.time.LocalDateTime.now());  
    }  
}
```

```

void playMusic() {
    System.out.println("Playing Music");
}

}

public class Main {

    public static void main(String[] args) {

        Car car = new Car();

        car.startEngine();
        car.playMusic();
        car.stopEngine();

    }

}

```

```

* Engine Started
Engine Started at: 03:46:49.2//509645
Playing Music
Engine Stopped

=== Code Execution Successful ===

```

### Explanation:

- The engine starts normally.
- The current time is then displayed.
- The original functionality is preserved

### Question 3: How can the parent class implementation still be executed?

#### Answer

- The parent class method can be executed using the **super keyword** .  
super.startEngine();

### Explanation:

- super refers to the immediate parent class.
- It executes the original Vehicle implementation before executing the additional code written inside Car.

### Question 4: What happens if Car does not override startEngine()?

#### Answer

If Car does not override startEngine():

- The method is inherited directly from Vehicle.
- The engine starts successfully.
- The engine start time is **not logged**, because no additional functionality is added.

## 2. Multilevel Inheritance – Scenario Scenario:

A software company models employees as follows:

Employee → stores employee ID and name.

Developer extends Employee → stores programming language.

SeniorDeveloper extends Developer → stores team size and performs code reviews.

Now HR requests that every SeniorDeveloper should be able to access employee

information and programming language details without duplicating code. Questions:

Class Diagram

Employee

|

Developer

|

SeniorDeveloper

### Q1. Why is multilevel inheritance suitable here?

Because each class adds new features while inheriting the previous ones.

- **Employee** → Employee ID, Name
- **Developer** → Programming Language
- **SeniorDeveloper** → Team Size, Code Reviews

This avoids duplicate code and improves reusability.

### Q2. Which members are accessible in SeniorDeveloper ?

SeniorDeveloper inherits all accessible members from both parent classes.

From **Employee**

- Employee ID
- Name
- Employee methods

From **Developer**

- Programming Language
- Developer methods

Own members

- Team Size
- Code Review methods

### Q3. If Developer overrides a method from Employee, which implementation will SeniorDeveloper inherit?

- SeniorDeveloper inherits the **Developer's overridden version** .

**Example:**

```
class Employee {  
    void display() {  
        System.out.println("Employee Details");  
    }  
}
```

```

class Developer extends Employee {
    @Override
    void display() {
        System.out.println("Developer Details");
    }
}

class SeniorDeveloper extends Developer {
    void codeReview() {
        System.out.println("Performing Code Review");
    }
}

public class Main {
    public static void main(String[] args) {
        SeniorDeveloper sd = new SeniorDeveloper();

        sd.display();
        sd.codeReview();
    }
}

```



The screenshot shows an IDE window with a file named 'Main.java'. The code in the editor is as follows:

```

20 }
21 }
22
23 public class Main {
24
25     public static void main(String[] args) {
26
27         SeniorDeveloper sd = new SeniorDeveloper();
28
29         // Calls the overridden method from Developer
30         sd.display();
31
32         // Calls SeniorDeveloper's own method
33         sd.codeReview();
34     }
35 }

```

On the right side of the IDE, there is an 'Output' panel. It displays the following text:

```

Developer Details
Performing Code Review

=== Code Execution Successful ===

```

**Question 4: Draw the inheritance hierarchy.**

Employee

Developer

SeniorDeveloper

### 3. Hierarchical Inheritance – Scenario Scenario:

An online shopping application has a base class Payment. Three classes inherit from it:

CreditCardPayment

UPIPayment

NetBankingPayment Each payment type processes transactions differently but shares common validation logic.

#### Q1. Why is hierarchical inheritance appropriate here?

Because multiple classes share the same parent but implement different behaviors.

All payment methods require:

- Validation
- Payment ID
- Amount

But transaction processing differs.

#### Q2. Where should common validation logic be implemented?

Inside the **Payment** class.

Example:

```
class Payment {  
  
    void validate() {  
        System.out.println("Payment Validated");  
    }  
}
```

#### Q3. How can each payment class provide its own transaction processing?

Each child class overrides processPayment().

```
class CreditCardPayment extends Payment {  
    void processPayment() {  
        System.out.println("Processing Credit Card Payment");  
    }  
}  
  
class UPIPayment extends Payment {  
    void processPayment() {  
        System.out.println("Processing UPI Payment");  
    }  
}
```

```

class NetBankingPayment extends Payment {
    void processPayment() {
        System.out.println("Processing Net Banking Payment");
    }
}

```

```

Main.java
1 class Payment {
2
3     void validate() {
4         System.out.println("Payment Validated");
5     }
6
7     void processPayment() {
8         System.out.println("Processing Payment");
9     }
10 }
11
12 class CreditCardPayment extends Payment {
13
14     @Override
15     void processPayment() {
16         System.out.println("Credit Card Payment Processed");

```

**Output**

```

Credit Card Payment:
Payment Validated
Credit Card Payment Processed

UPI Payment:
Payment Validated
UPI Payment Processed

Net Banking Payment:
Payment Validated
Net Banking Payment Processed

Code Execution Successful

```

#### Question 4: What is the advantage over writing three independent classes?

##### Answer

Advantages include:

- Code reuse
- Reduced duplication
- Easy maintenance
- Better scalability
- Easier future updates

For example, if validation logic changes, it only needs to be updated in the Payment class instead of all three child classes.

#### 4. Compile-Time Polymorphism (Method Overloading) –

Scenario Scenario: A calculator application supports addition of: Two integers Three integers Two decimal numbers The product owner wants users to call the same method name (add) regardless of the input.

##### Scenario

A calculator supports addition of:

- Two integers
- Three integers
- Two decimal numbers

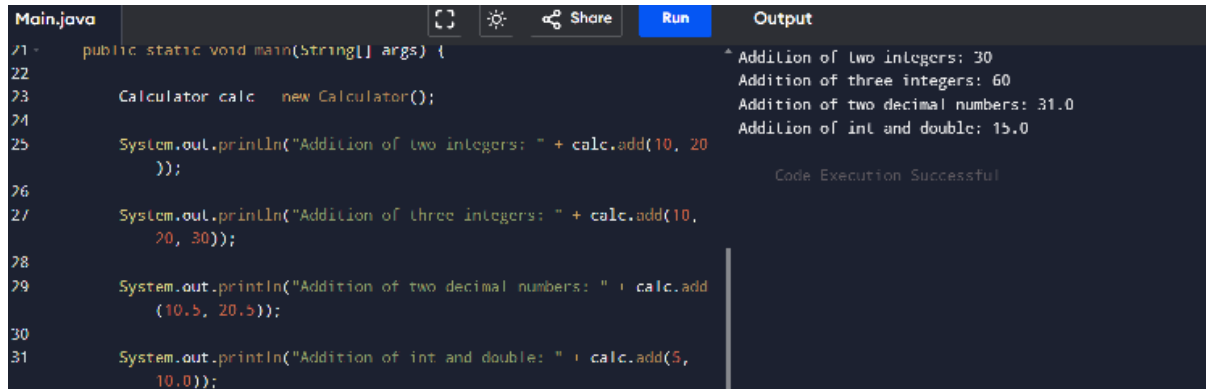
The same method name (add) should be used for all operations.

#### Question 1: Which OOP concept should be used?

##### Answer

The appropriate concept is **Method Overloading (Compile-Time Polymorphism)** .

### Example



```
Main.java
21 public static void main(String[] args) {
22
23     Calculator calc = new Calculator();
24
25     System.out.println("Addition of two integers: " + calc.add(10, 20));
26
27     System.out.println("Addition of three integers: " + calc.add(10, 20, 30));
28
29     System.out.println("Addition of two decimal numbers: " + calc.add(10.5, 20.5));
30
31     System.out.println("Addition of int and double: " + calc.add(5, 10.0));
}
```

Output

```
^ Addition of two integers: 30
Addition of three integers: 60
Addition of two decimal numbers: 31.0
Addition of int and double: 15.0
Code Execution Successful
```

### Question 2: How will Java decide which method to invoke?

Answer

Java selects the appropriate overloaded method during compile time based on:

- Number of parameters
- Data types of parameters
- Order of parameters

### Question 3: Can methods be overloaded by changing only the return type?

Answer

No.

Methods cannot be overloaded by changing only the return type because Java identifies overloaded methods using the method signature (method name + parameter list), not the return type.

### Question 4: What happens if `add(5, 10.0)` is called?

Answer

Java automatically converts the integer 5 into a double and calls:

`add(double, double)`

This process is known as automatic type promotion.

## 5. Runtime Polymorphism (Method Overriding)

### Scenario

A food delivery application contains:

Restaurant



Pizza Burger Biryani

Each restaurant calculates delivery time differently.

The application stores all restaurants using:

List<Restaurant> restaurants;

and calls:

restaurant.calculateDeliveryTime();

**Question 1: Which OOP concept is being used?**

**Answer**

The application uses **Runtime Polymorphism (Method Overriding)** .

**Question 2: Which version of calculateDeliveryTime () executes?**

**Answer**

The version belonging to the **actual object** executes.

For example:

Restaurant r = new Pizza();

r.calculateDeliveryTime();

Output:

Pizza Delivery Time

**Question 3: When is the method selection decided?**

**Answer**

The method selection is decided **during runtime** .

This is known as **Dynamic Method Dispatch**, where Java determines the actual object type while the program is running.

**Question 4: What would happen if calculateDeliveryTime () were declared static?**

**Answer**

Static methods cannot be overridden.

Instead, Java calls the method based on the **reference type** , not the object type.

Therefore, runtime polymorphism would not occur.

**Scenario**

A smart home application supports:

- Smart Bulb

- Smart Fan
- Smart AC

All devices can:

- Turn On
- Turn Off

Each device implements these operations differently.

Third-party manufacturers should also be able to add new devices without changing existing code.

### **Question 1: Would you use an interface or inheritance? Why?**

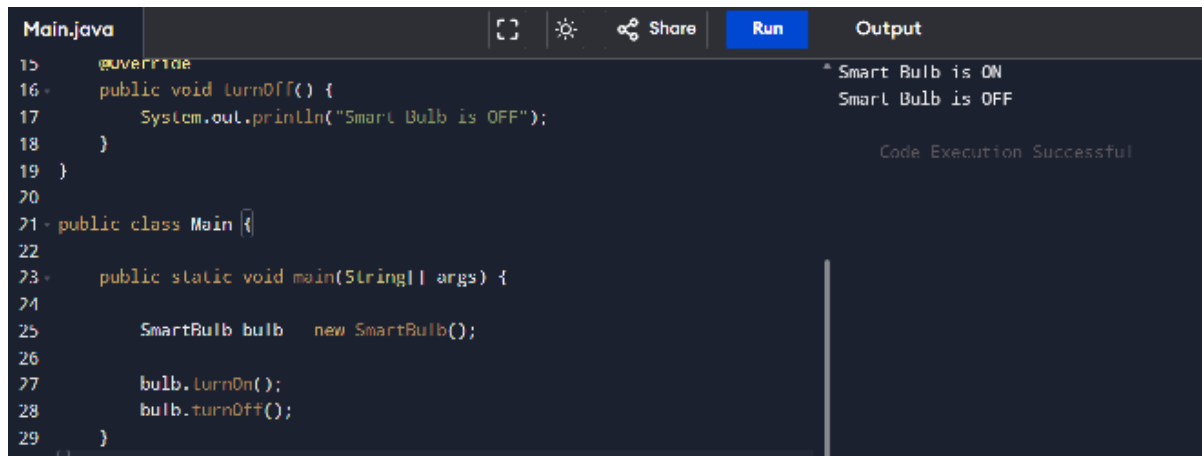
#### **Answer**

An **interface** is the best choice because:

- All devices share the same behavior (turnOn() and turnOff()).
- Each device has its own implementation.
- New devices can be added easily by implementing the interface.
- Existing code does not need to be modified.

### **Question 2: Design an appropriate interface.**

```
interface SmartDevice {  
    void turnOn();  
    void turnOff();  
}  
  
class SmartBulb implements SmartDevice {  
    public void turnOn() {  
        System.out.println("Bulb ON");  
    }  
    public void turnOff() {  
        System.out.println("Bulb OFF");  
    }  
}
```



The screenshot shows a Java IDE with a file named 'Main.java'. The code defines a 'SmartBulb' class with 'turnOn()' and 'turnOff()' methods, and a 'Main' class with a 'main' method that creates a 'SmartBulb' object and calls these methods. The output pane on the right shows the execution results: 'Smart Bulb is ON' and 'Smart Bulb is OFF', followed by 'Code Execution Successful'.

```
Main.java
15  @Override
16  public void turnOff() {
17      System.out.println("Smart Bulb is OFF");
18  }
19  }
20
21  public class Main {
22
23      public static void main(String[] args) {
24
25          SmartBulb bulb = new SmartBulb();
26
27          bulb.turnOn();
28          bulb.turnOff();
29      }
30  }
```

Output

```
* Smart Bulb is ON
Smart Bulb is OFF

Code Execution Successful
```

**Question 3: Can one device implement multiple interfaces? Give an example.**

**Answer**

**Yes.**

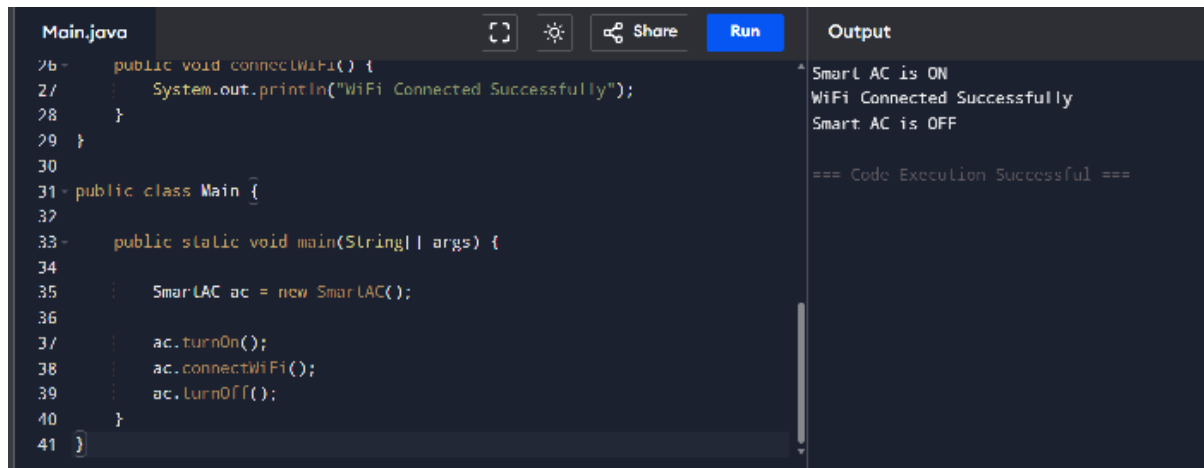
A class can implement multiple interfaces.

Example:

```
interface SmartDevice {
    void turnOn();
    void turnOff();
}
```

```
interface WiFiEnabled {
    void connectWiFi();
}
```

```
class SmartAC implements SmartDevice, WiFiEnabled {
    public void turnOn() {
        System.out.println("AC ON");
    }
    public void turnOff() {
        System.out.println("AC OFF");
    }
    public void connectWiFi() {
        System.out.println("WiFi Connected");
    }
}
```



```
Main.java
26- public void connectWiFi() {
27-     System.out.println("WiFi Connected Successfully");
28- }
29- }
30-
31- public class Main {
32-
33-     public static void main(String[] args) {
34-
35-         SmartAC ac = new SmartAC();
36-
37-         ac.turnOn();
38-         ac.connectWiFi();
39-         ac.turnOff();
40-     }
41- }
```

```
Output
Smart AC is ON
WiFi Connected Successfully
Smart AC is OFF

=== Code Execution Successful ===
```

**Question 4: Why is an interface better than creating a common base class in this scenario?**

**Answer**

An interface is better because:

- It promotes **loose coupling**, allowing devices to be independent of one another.
- It supports **multiple interface implementation**, enabling a device to have several capabilities (such as SmartDevice and WiFiEnabled).
- It allows **easy integration of third-party devices** without modifying existing classes.
- It focuses on defining **common behavior** rather than forcing a shared implementation.
- It improves the **flexibility, scalability, and maintainability** of the smart home application.

## Conclusion

These scenarios demonstrate the core Object-Oriented Programming concepts in Java:

- **Single Inheritance** – Enables a child class to inherit features from one parent class.
- **Multilevel Inheritance** – Allows a class to inherit through multiple levels, promoting code reuse.
- **Hierarchical Inheritance** – Multiple child classes share common functionality from one parent class.
- **Compile-Time Polymorphism** – Achieved using method overloading, where the compiler selects the appropriate method.
- **Runtime Polymorphism** – Achieved using method overriding, where the JVM selects the method based on the actual object.
- **Interface** – Defines common behaviors while allowing different classes to

provide their own implementations, making applications flexible and extensible.